Stoquify Enterprise Table Benchmark and Audit Report

Date: 2026-08-28
Scope: Research, audit, comparison, and recommendations only
Product code changed: None
Evidence bundle: `what-next/ui-ux/evidence/table-system-benchmark-2026-08-28/`

Executive recommendation

Stoquify should evolve its existing TanStack-based table foundation instead of adopting a broad replacement grid. The current system already has credible primitives for sorting, filtering, pagination, visibility, robust route states, bilingual controls, and permission/module gates. Its enterprise gap is not primarily rendering technology. It is the absence of a governed table product contract across modules.

The highest-value move is to define one canonical table shell with four explicit contracts: server-owned query state, user/role saved views, persistent column and density preferences, and auditable selection/bulk-action behavior. Pilot that contract on inventory items, purchase orders, and payment reconciliation. Delay virtualization and any premium-grid decision until measured datasets demonstrate that the shared stack cannot meet a documented performance envelope.

Weighted result: Stoquify scores **61.9/100** against a benchmark leader range of **77.2-91.5/100**. Its strongest dimension is enterprise security/audit boundary design. Its weakest dimensions are column personalization, mobile table adaptation, and consistent search/filter/saved-view behavior.

Evidence and claim policy

The report uses three claim classes:

- **Documented capability**: directly supported by a cited primary source.
- **Repository observation**: directly supported by inspected Stoquify source, route catalogs, tests, prior inventories, or graph artifacts.
- **Assessment**: reviewer judgment derived from documented capability and repository observation; it is not a certification or production benchmark.

No platform was accessed through a production tenant. No authenticated competitor screenshots were captured. Visual, accessibility, mobile, and performance scores therefore carry lower confidence than feature-presence scores. The screenshot limitation is recorded in `screenshot-manifest.json`.

Why these seven platforms

The selection rubric required exactly seven products, at least four broad ERP suites, one inventory/POS leader, one HRIS/payroll leader, and one finance/reconciliation leader. Products were selected for relevance to Stoquify's multi-module operating-system ambition, public primary documentation, and identifiable table/workbench patterns.

Platform	Segment role	Selection reason
Odoo 19	Broad ERP suite	Broad SMB/mid-market record-list model with search, filters, favorites, mass editing, and configurable list views.
Microsoft Dynamics 365 Finance and Operations	Broad ERP suite	Deep enterprise grid productivity, saved views, freezing, grouping, calculations, and Excel interoperability.
SAP S/4HANA Cloud / SAP Fiori	Broad ERP suite	Most explicit public table design system across responsive, grid, and analytical archetypes.
Oracle NetSuite	Broad ERP suite	Mature saved-search, configurable-list, role, export, and finance-oriented evidence workflows.
Shopify Admin / POS	Inventory/POS specialist	Strong resource-list search, saved views, bulk operations, mobile reach, and operational simplicity.
Workday	HRIS/payroll specialist	Enterprise HR analytics, worksheets, governance, auditability, lineage, and cross-device reporting.
Stripe Dashboard	Finance/reconciliation specialist	High-quality payment search, financial reporting, payout reconciliation, and export semantics.

Scoring model

Each dimension is scored from 0 to 5. Weighted contribution is `raw score / 5 * weight`. Weights total 100.

Dimension	Weight
Visual polish and coherence	12
Density and readability	12
Search, filter, and saved views	12
Column personalization	10
Selection, bulk actions, inline editing, and recoverability	12
Loading, empty, error, and permission states	8
Accessibility and keyboard semantics	10
Mobile and responsive adaptation	6
Enterprise security and audit controls	8
Performance and high-volume behavior	5
Export, reconciliation, and evidence workflows	5

Weighted scorecard

Rank	Platform	Score / 100	Evidence confidence	Benchmark interpretation
1	SAP S/4HANA Cloud / Fiori	91.5	Medium-high	Best governed archetype and personalization model.
2	Microsoft Dynamics 365	89.3	Medium-high	Best dense grid productivity and data-entry ergonomics.
3	Stripe Dashboard	88.6	Medium	Best finance search/reconciliation/evidence clarity.
4	Shopify Admin / POS	88.1	Medium-high	Best task-focused saved-view and bulk-work simplicity.
5	Workday	84.6	Medium	Strong governed analytics, accessibility investment, and HR data lineage.
6	Odoo	82.1	Medium-high	Broad, pragmatic ERP list behavior with mass editing and favorites.
7	Oracle NetSuite	77.2	Medium-high	Deep saved-search/export controls, offset by older interaction density.
8	Stoquify current state	61.9	High for repository features; low for runtime UX	Strong control boundaries, fragmented presentation contract.

The machine-readable scorecard contains every raw value, weight, formula, total, and confidence note. Scores are comparative assessments, not vendor certifications or measured production SLAs.

Platform findings

1. Odoo 19

Documented strengths:

- A unified search field supports field-aware search, preconfigured and custom filters, grouping, private/shared favorites, and default favorites. [ODOO-01]
- List views support simple inline editing, optional mass editing, default sorting/grouping, action buttons, and explicit create/edit/delete/duplicate capability controls. [ODOO-02]
- Access rights and record rules distinguish read, write, create, and delete behavior at role/group/model scope. [ODOO-03]

Best-in-class pattern: **one composable search grammar across many business objects**. The favorite is not just a filter bookmark; it can be shared and made default.

Weaknesses and limits: Mobile can prefer Kanban over list view, which is a sensible adaptation but weakens cross-device table consistency. Public documentation does not establish latency, keyboard completion, or transactional undo guarantees.

Stoquify lesson: Adopt Odoo's compact filter/group/favorite mental model, but preserve Stoquify's stricter service-owned business truth and explicit permission/module-denial states.

2. Microsoft Dynamics 365 Finance and Operations

Documented strengths:

- Grid capabilities include calculated values, type-ahead entry, mathematical expressions, grouping, frozen columns, auto-fit, stretchable columns, and Excel export. [D365-01]
- Personalization can persist automatically or through saved views; organizations can publish defaults and optimize views for data entry. [D365-02]

Best-in-class pattern: **dense grids are treated as user-configurable workspaces, not passive tables**. Column freezing, calculations, grouping, and saved layouts support sustained operational use.

Weaknesses and limits: Capability depth creates discoverability and governance costs. Mobile evidence for the same dense grid experience is weaker than desktop evidence. Public docs describe features, not end-to-end recovery from failed bulk edits.

Stoquify lesson: Add a constrained enterprise mode for data-entry-heavy tables, with role-governed defaults and personal views. Do not expose every grid feature on every archetype.

3. SAP S/4HANA Cloud / SAP Fiori

Documented strengths:

- Responsive tables prioritize key columns and move secondary data into labeled pop-ins on narrow screens. Toolbars can expose variant management, view settings, personalization, and actions. [SAP-01]
- Personalization explicitly covers visibility, order, sorting, grouping, and filtering, with different patterns for simple and complex tables. [SAP-02]
- Analytical tables support large row counts, grouping, aggregations, frozen columns, persisted settings, and a deliberate adaptive alternative for smartphones. [SAP-03]

Best-in-class pattern: **table archetype selection is a design-system decision**. Responsive, analytical, and other table types have distinct use criteria instead of one component pretending to fit all workloads.

Weaknesses and limits: The analytical table is explicitly not fully responsive and requires another smartphone UI. Personalization can become complex, and SAP warns against redundant filtering controls.

Stoquify lesson: Formalize archetypes and adaptive mobile alternatives. Use one visible filter authority per surface. Persist layout only where the job benefits from it.

4. Oracle NetSuite

Documented strengths:

- Custom list views can change displayed columns and footer filters; saved searches can become selectable list views. [NETSUITE-01]

- Search results export to CSV, PDF, and Excel subject to permissions, and saved-search exports are recorded in an execution log. [NETSUITE-02]
- Saved searches support criteria, result-field selection, available filters, save/run, and CSV validation workflows. [NETSUITE-03]

Best-in-class pattern: **saved search as governed operational artifact**. Search definitions can power lists, reports, exports, schedules, and audit history.

Weaknesses and limits: The interaction model is administratively powerful but visually older and more procedural. Export documentation highlights spreadsheet precision risks. Mobile table depth is not strongly evidenced.

Stoquify lesson: Treat saved views and exports as permissioned, traceable assets. Include export scope, filter snapshot, requester, generated-at time, locale/currency context, and integrity metadata.

5. Shopify Admin / POS

Documented strengths:

- Resource lists support default and custom views, autocomplete filter suggestions, `is/is not` and `and/or` logic, sorting, search, pagination, and resource-specific bulk actions. [SHOPIFY-01]
- Product views persist filters and selected columns, update membership automatically as data changes, and integrate directly with bulk editing. [SHOPIFY-02]

Best-in-class pattern: **saved views are task shortcuts**. The interface keeps advanced logic compact while connecting filtered result sets directly to bulk work.

Weaknesses and limits: View management is desktop-only even though views are consumable on mobile. Some sort persistence differs by resource. Audit/evidence semantics are lighter than finance ERP systems.

Stoquify lesson: Make common task views first-class: low stock, awaiting supplier, unmatched payments, blocked payroll, overdue receivables. Keep filters readable as tokens and make bulk scope explicit.

6. Workday

Documented strengths:

- The reporting platform includes Prism Analytics, Discovery Boards, People Analytics, custom reporting, Report Designer, Slides, and Worksheets. [WORKDAY-01]
- Prism documentation describes reporting/worksheet outputs, desktop/tablet/mobile access, governance, auditability, and data lineage. [WORKDAY-02]

Best-in-class pattern: **sensitive workforce tables sit inside governed analytics and lineage**, not as isolated generic grids.

Weaknesses and limits: Public sources expose less row-level interaction detail than SAP, Dynamics, or Shopify. Some advanced analytics capabilities may depend on product packaging and configuration.

Stoquify lesson: HRIS/payroll saved views and exports must preserve own-record/manager/admin scope, redaction, effective-date context, and evidence lineage. Payroll permission must not implicitly unlock HRIS truth.

7. Stripe Dashboard

Documented strengths:

- Dashboard search spans payments, customers, invoices, payouts, connected accounts, and products, with field filters, operators, dates, negation, identifiers, and sortable result groups. [STRIPE-01]
- Reports cover balance, payout reconciliation, bank reconciliation, fees, and balance changes, with Dashboard views and downloadable CSV. [STRIPE-02]
- Report configuration supports account subsets, currency/report-category filters, and programmatic report runs for reconciliation automation. [STRIPE-03]

Best-in-class pattern: **search and reconciliation share a financial-object vocabulary**. Users can move from global discovery to a filtered, exportable, account-scoped report without losing semantic context.

Weaknesses and limits: Column personalization is intentionally lighter than ERP grids. High-risk bulk mutation is not the center of the experience, so it is a weaker reference for inventory mass editing.

Stoquify lesson: Standardize financial identifiers, account/tender/currency/date filters, and recon status across finance tables. Every reconciliation export should carry the exact query and account scope that produced it.

Stoquify repository audit

Quantitative baseline

The 2026-08-23 repository inventory found:

- 57 table-bearing files and 74 table occurrences.
- 7 shared `DataTable` adopters versus 35 files with native or bespoke table markup.
- 37 toolbar-like surfaces, 32 pagination patterns, 11 search-like patterns, and 9 date-range patterns.
- 44 files with explicit minimum-width patterns, indicating widespread horizontal-scroll dependence.
- 2 files with explicit `aria-sort` at that audit point. The shared table now exposes `aria-sort`, but module consistency remains unproven without a browser pass.

Category distribution was finance/accounting 14, payroll/HRIS 14, inventory 8, purchasing 8, administration 4, POS 2, and other 7. These counts are source-level occurrences, not unique end-user routes.

Current strengths worth preserving

- The shared TanStack wrapper already supports sorting, column filtering, pagination, visibility, and row-model composition.
- `DataTablePagination` and `DataTableViewOptions` provide a credible base for a governed shell.
- `TableDateRangePicker` has begun normalizing history surfaces across inventory, purchasing, finance, POS, dashboard history, and payroll controls.
- Inventory items and purchase orders demonstrate server/query-param entry with a paginated initial result rather than unconditional full-dataset loading.
- Payment reconciliation, people approvals, cash drawer, and specialized finance ledgers already use domain-specific workbenches rather than generic CRUD-only lists.
- Route catalogs enforce permissions and, where applicable, commercial-module entitlements. Examples include `inventory.items.read`, `purchases.orders.read`, `payments.reconciliation.read`, `hris.people.read`, `payroll.employees.read`, `accounting.journal.read`, and `OPERATE_POS`.

- People access explicitly prevents payroll permissions from unlocking HRIS people truth, an important privacy boundary.
- Many routes provide deliberate loading, empty, permission-denied, and module-locked states in English and French.
- Table contract tests and route tests exist, including a system table-presentation contract and purchase-order analytics sorting/accessibility coverage.

Systemic gaps

Gap	Repository observation	Business/control impact	Priority
Fragmented implementation	Shared wrapper adoption is a minority of table-bearing files.	Users relearn filters, pagination, actions, and responsive behavior; fixes do not propagate.	Critical
Ambiguous query ownership	Client filtering, server pagination, and hybrid initial slices coexist.	Counts, selection scope, exports, and URLs can disagree.	Critical
No governed saved-view model	Visibility is mostly local component state; shared role views are not a platform contract.	Repeat work, inconsistent operational queues, no administrator-published defaults.	High
Weak column persistence	Column visibility exists, but durable order, width, pinning, and role defaults are not established.	Dense finance/payroll users cannot stabilize their workspace.	High
Bulk-action semantics vary	Selection and action placement are surface-specific.	Risk of acting on visible page versus all matching records; weak recovery and evidence.	Critical
Horizontal-scroll-first mobile	Many tables rely on minimum widths and overflow.	Mobile users lose context and action reachability.	High
Accessibility coverage is partial	Shared `aria-sort` is promising, but native tables and full keyboard flows remain uneven.	Screen-reader and keyboard users receive inconsistent state/action feedback.	Critical
Performance contract absent	No common thresholds for server mode, virtualization, row counts, or filter latency.	High-volume routes may ship with accidental client work and memory cost.	High
Export evidence varies	Finance/reconciliation concepts are strong, but export provenance is not one table-system contract.	Auditors cannot uniformly reproduce result sets.	High
Localization drift	Some shared and payroll controls are bilingual; table labels and pagination semantics are not uniformly governed.	EN/FR parity and locale-safe number/date behavior can regress.	Medium

Module heatmap

Scores are 0-5 repository assessments. `Security` includes route permission/module boundary quality, not penetration-test results.

Module family	Consistency	Query/views	Actions/recovery	A11y/mobile	Security	Evidence/export	Main finding
Inventory	3.2	3.1	3.0	2.9	4.2	3.1	Strong route coverage; mixed table shells and mobile overflow.
Purchasing/AP	3.3	3.2	3.2	3.0	4.3	3.5	Good shared-table and analytics precedents; saved views and bulk scope need governance.
Finance/reconciliation	3.6	3.5	3.4	3.0	4.6	4.2	Strongest proof semantics; specialized surfaces need shared interaction contracts.
Accounting/close	3.3	3.0	3.1	2.8	4.6	4.1	Good permission taxonomy; statement and ledger archetypes need separate layouts.
Payroll/HRIS	3.4	3.0	3.2	3.0	4.7	3.8	Strong privacy/module gates and bilingual patterns; high density needs role views and redaction-aware export.
POS/cash drawer	3.5	3.1	3.5	3.2	4.2	3.6	Operational clarity is strong; terminal/mobile table behaviors remain bespoke.
Sales/receivables	3.0	2.9	2.9	2.7	4.0	3.1	Needs queue-oriented views tied to cash impact.
Compliance/assurance	3.3	3.1	3.1	2.8	4.6	4.3	Evidence-rich domain; should adopt immutable evidence-table archetype.

Module family	Consistency	Query/views	Actions/recovery	A11y/mobile	Security	Evidence/export	Main finding
Admin/reference data	3.1	2.7	3.0	2.8	4.1	2.8	Best candidate for simple master-data archetype and shared wrapper migration.

Architecture and graph evidence

Existing `graphify-out` artifacts were used for dependency and impact reasoning. The previous graph review identified the shared `DataTable` and `UnitsManagementDashboard` in a small/isolated community, `EnhancedBrandsManagement` in another community, and `DataTablePagination` in a much larger shared community. This supports a fragmentation finding: pagination is broadly connected, while the table composition layer is not yet the dominant module boundary.

The route graph and route-data-access catalogs also show that table surfaces are downstream of permission and module gates. Harmonization must therefore stay presentation-focused: tables may consume trusted read models and call protected actions, but must not reimplement tenant, permission, entitlement, redaction, accounting, inventory, payroll, or reconciliation truth in client state.

Responsive and mobile assessment

Stoquify's common fallback is horizontal scrolling with minimum table widths. That is acceptable for analytical/ledger tables on desktop but should not be the default mobile strategy. Use three deliberate adaptations:

- Preserve a compact scrollable grid for expert desktop/tablet workflows where cell relationships matter.
- Use priority columns plus labeled row detail/pop-in for master data and queues.
- Replace the table with cards, a detail list, or an action queue on phones when row identity and next action matter more than cell comparison.

Accessibility assessment

The shared `aria-sort` implementation and explicit sortable-header precedents are positive. A release-grade table contract still needs keyboard sorting, focus retention after paging/filtering, announced result counts, selection state, bulk-action scope, error association, accessible menus, visible focus, reduced-motion safety, and screen-reader verification across native and shared adopters. This report does not claim WCAG conformance.

Performance assessment

No production dataset or browser trace was run for this research-only task. Repository evidence shows both server/query-param and client-row-model approaches. A performance contract is needed before choosing virtualization:

- Default server mode when filtered result sets or authorization-safe counts cannot be bounded cheaply in the browser.
- Require stable sort keys and cursor or offset semantics with deterministic tie-breakers.
- Keep selection by durable identifier, never visible row index.
- Export through a server-owned job/query snapshot rather than serializing the current DOM.

- Trial virtualization only on measured high-volume analytical surfaces; it complicates accessibility, variable row height, printing, and evidence capture.

Canonical Stoquify table archetypes

Archetype	Typical modules	Required behavior	Mobile behavior
Master/reference list	Items, suppliers, customers, accounts, employees, units	Search, filters, saved views, column visibility, row actions, safe bulk operations	Priority columns with row detail or cards
Transaction work queue	Purchase orders, approvals, receivables, payroll readiness	Status/task views, ownership, due/risk, selection scope, guarded actions, recovery	Action list with expandable proof
Finance statement	Trial balance, P&L, cash flow	Frozen identity columns, numeric alignment, totals, period/entity/currency context, export provenance	Summary first; drill to line detail
Analytical/high-volume grid	Sales, inventory analysis, finance analytics	Server query, grouping, aggregations, optional pinning/virtualization, reproducible export	Alternate summary/filter UI
Evidence/audit ledger	Reconciliation, movement history, assurance, audit history	Immutable identifiers, event time, actor/source, status, proof link, append-only interpretation	Timeline or compact evidence list
Editable line-item grid	Orders, journals, controlled setup	Validation, keyboard entry, totals, dirty-state protection, permissioned commit, reversible drafts	Dedicated form/stepper, not compressed spreadsheet
Bounded detail table	Detail page subtables	Small fixed result set, clear empty state, minimal controls	Stacked labeled fields

Patterns to adopt

- SAP: explicit archetypes, responsive pop-ins, persisted personalization, and one filter authority.
- Dynamics: role/user views, frozen columns, dense data-entry mode, and column calculations where domain-safe.
- Shopify: readable filter tokens, saved task views, and bulk actions directly tied to the current result set.
- Odoo: shared/private favorites, default views, grouping, and mass-edit enablement controlled per view.
- NetSuite: permissioned exports, saved-search execution trace, and reusable governed search definitions.
- Workday: redaction-aware HR analytics, effective-date context, governance, lineage, and cross-device alternatives.
- Stripe: consistent financial-object search, account/currency/date scope, and reconciliation-specific export semantics.

Patterns to avoid

- One universal table configuration for every archetype.

- Simultaneous toolbar filters and hidden column-menu filters with divergent state.
- Selection copy that does not distinguish current page, loaded rows, and all matching records.
- Client-only filtering over a server-paginated subset.
- Persisting column state without schema/version migration.
- Inline editing for posted journals, reconciled evidence, payroll results, or other controlled facts.
- Exports that omit filters, sort, locale, currency, entity, time zone, requester, and generated-at metadata.
- Virtualization as a default before real performance evidence.
- Horizontal scroll as the only phone adaptation.
- Visual status conveyed only by color or unlabeled icons.

Reviewer-lens disposition

All requested reviewer lenses were material. The detailed register is in `reviewer-lens-register.json`.

Lens	Material conclusion
Principal architecture	Evolve one table engine behind archetype contracts; preserve server-owned truth.
Frontend engineering	Consolidate wrapper, toolbar, pagination, query adapter, preference adapter, and state components.
Product and UX	Design around jobs and saved task views, not feature parity with premium grids.
Accounting/controls	Separate draft editing from posted evidence; make exports reproducible and traceable.
Security/privacy	Keep route/RBAC/module/redaction gates authoritative; never infer authorization from visible columns.
Accessibility	Treat keyboard/screen-reader semantics as an acceptance gate across every archetype.
Mobile	Provide adaptive alternatives by archetype, not uniform horizontal scroll.
Performance/data	Establish server-mode and virtualization thresholds from measured datasets.
QA/governance	Add table contract tests, route smoke, screenshots where authorized, and migration inventory ratchets.
Growth/customer success	Saved role views and coherent cross-module behavior reduce training and improve enterprise trust.

Unresolved questions

- Which three roles receive administrator-published default views first: accountant, stockkeeper, purchasing manager, payroll admin, or owner?
- Should personal table preferences synchronize across devices or remain browser-local for the first release?
- What datasets and latency budgets define server mode, client mode, and virtualization eligibility?
- Which bulk actions require fresh authentication, dual control, reason codes, or asynchronous jobs?

- What retention and legal-evidence rules apply to saved views, export manifests, and reconciliation extracts by country pack?
- Which authenticated routes and seeded tenants may be used for screenshot and assistive-technology verification?

Evidence limitations

- Competitor research used public documentation, not authenticated production tenants.
- No third-party product was load-tested or independently accessibility-tested.
- Public documentation can describe intended behavior without proving every deployment implements it identically.
- Stoquify runtime screenshots were not captured because this research task did not receive authenticated demo credentials or an approved seeded workspace.
- The earlier route smoke artifact covers public routes and does not establish authenticated table behavior.
- Performance scores are directional assessments; no production trace, bundle profile, or large-dataset benchmark was run.

Decisions requiring approval

- Approve TanStack evolution as the default architecture and require an explicit exception record for another grid engine.
- Approve the seven canonical archetypes and their mobile adaptations.
- Approve server-owned saved views with user, role, organization, schema version, and permission-safe visibility scopes.
- Approve the first three pilots: inventory items, purchase orders, and payment reconciliation.
- Approve release gates for keyboard/screen-reader behavior, query/selection correctness, export provenance, EN/FR parity, and responsive adaptation.

Recommended next implementation prompt

```
Act as Stoquify's principal frontend, product, accessibility, security, and financial-controls team. Implement only the NOW slice of the 2026-08-28 table harmonization roadmap. Evolve the existing TanStack-based shared table stack; do not introduce a second grid library. Define the canonical table contract, URL/server query adapter, selection-scope model, persistent versioned column/density preferences, permission-safe saved-view schema, robust state semantics, and audit-safe export request contract. Pilot the contract on inventory items, purchase orders, and payment reconciliation while preserving service-owned business truth, tenant isolation, RBAC, module entitlement, redaction, bilingual EN/FR behavior, and existing protected actions. Add focused contract tests and authenticated desktop/tablet/mobile evidence only where credentials and seeded data are approved. Do not migrate unrelated tables.
```

Sources

The source name, publisher, publication/update date when stated, access date, URL or local path, claim supported, and limitations are recorded in `source-manifest.json`.